

Array (Dizi) Yapısı

İçerik

- [Arrays](#)
- [Dynamic Array](#)
- [Dynamic Array with Python](#)
- [Array İş Mülakatları Soru-Cevap](#)
- [Array Python Challenge/Problem](#)
- [Neler Öğrendik](#)

Arrays

- 1 mesafe sensörü düşünün. Saniye de 1 veri yollayan bir sensör.
- Bu verileri depolamak için variable tanımlayabilir miyiz?
- Peki variable yerine ne kullanabiliriz? ARRAYS
- Value depolamamıza yarayan yapılara array diyoruz.
- Örn: [1,2,3,4,5] = higher level abstraction yani real world'de böyle kullanılır.
- Bu array 5 elemanlı bir array.
- Arrayin içinde ki her bir elemanın yerini indexler belirler.
- 2 sayısı yani değeri index 1 dedir.
- Biz tüm bu kursumuzda python kullandığımız gibi data structure konusunda da python kullanacağız ve python da index sıfırdan başlar.

In [1]:

```
import numpy as np
array = np.array([[1,2,3,4,5]]) # vector 1D
print(array)
print("Boyut: ",array.shape)
```

```
[[1 2 3 4 5]]
Boyut: (1, 5)
```

- Peki array tek boyulu olmak zorunda mı? Hayır 2 boyutlu hatta 3-4 boyutlu bile olabilir. (Optional: tensor is lower or higher dimensional array)
- 2D array satır ve sütunlardan oluşur. Row - Column

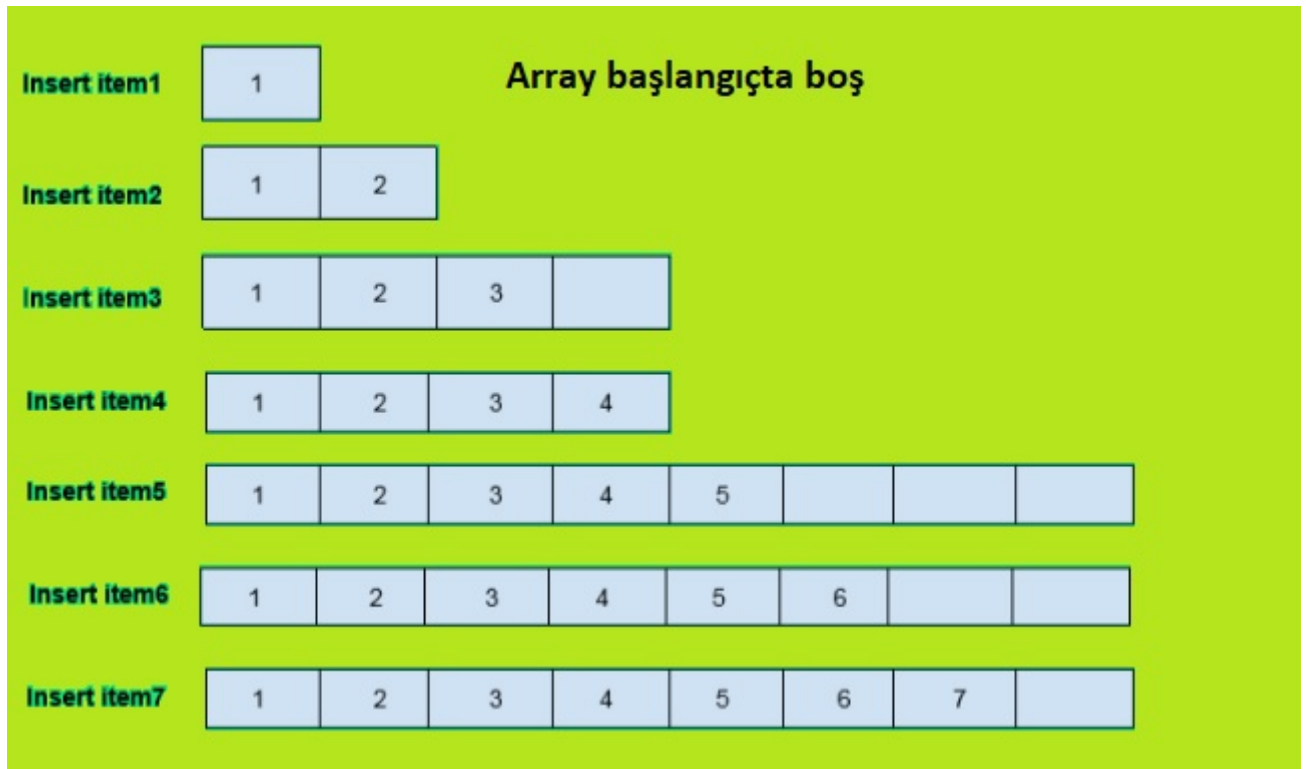
In [2]:

```
array2D = np.array([[1,2,3,4,5],  
                    [6,7,8,9,10]])  
print(array2D)  
print("boyut: ",array2D.shape)  
print("2. satır 4. sütun: ",array2D[1,3])
```

```
[[ 1  2  3  4  5]  
 [ 6  7  8  9 10]]  
boyut: (2, 5)  
2. satır 4. sütun: 9
```

Dynamic Array

- Dynamic array boyutunu önceden belirtmek zorunda olmadığımız daha sonra eleman ekleyip çıkartabileceğimiz yapılara denir.
- Dynamic array growable and resizable olarak tanımlanır.
- Dynamic array neredeyse tüm programlama dillerinde çok sık kullanılan bir yapıdır.



Dynamic Array with Python

- Anlamak için kendi dynamic array class'ımızı yazalım.
- Eğer class ile ilgili sorunlarınız var ise nesne yönelimli programlama notlarına bakınız.

In [3]:

```
import ctypes # yeni array yaratmak icin kullanacagiz

class DynamicArray(object):

    # initialize (constructor)
    def __init__(self):
        self.n = 0 # eleman sayisi
        self.capacity = 1 # kapasite
        self.A = self.make_array(self.capacity)

    def __len__(self):
        """
        return array icerisinde eleman sayisi
        """
        return self.n

    def __getitem__(self, k):
        """
        return index k'da ki eleman(value)
        """
        if not 0 <= k < self.n:
            return IndexError("k is out of bounds !")

        return self.A[k]

    def append(self, eleman):
        """
        array'e eleman ekler
        """

        # eger kapasite dolu ise kapasiteyi iki katina cikar
        if self.n == self.capacity:
            self._resize(2*self.capacity)

        self.A[self.n] = eleman # eleman ekleme
        self.n += 1 # eleman sayisi bir arttir

    def _resize(self, new_cap):
        """
        array kapasitesini arttir
        """

        B = self.make_array(new_cap) # yeni array yap

        # eski array (A) icerisindeki degerleri yeni arraye(B) icine tasi
        for k in range(self.n):
            B[k] = self.A[k]

        self.A = B # arrayi guncelle
        self.capacity = new_cap # kapasite guncelle

    def make_array(self, new_cap):
        """
        return yeni array
        """
        return (new_cap*ctypes.py_object)()
```

In [4]:

```
# obje tanımla
arr = DynamicArray()
# append new element 1
arr.append(1)
print(arr[0])
# append new element 1 , 3
arr.append(3)
print(arr[0],arr[1])
# append new element 1 , 3 ,5
arr.append(5)
print(arr[0],arr[1],arr[2])
```

```
1
1 3
1 3 5
```

Soru-Cevap

1. Dynamic Array neden önemli?

- Automatic resizing (Boyutunun azalıp artabilmesi)

2. Array ile Dynamic Array arasındaki fark nedir?

- Array is fixed size. Başlangıçta yani array yaratırken size belirlemek lazım.
- Dynamic array de başlangıçta size belirlemeye gerek yok.

3. Dynamic Array Advantages and Disadvantages ?

- Advantages:
 - Fast lookups: istenilen arrayde ki degeri elde etmesi. Yani istenilen indeksteki değeri elde etmesi $O(1)$.
 - Variable size: resizeable (Boyutunun azalıp artabilmesi)
- Disadvantages:
 - Slow worst-case appends: eger yer yoksa kapasiteyi arttırmak lazım ancak bu işlem nispeten yavaştır. $O(n)$
 - Costly inserts and deletes

	Average Case	Worst Case
space	$O(n)$	$O(n)$
lookup	$O(1)$	$O(1)$
append	$O(1)$	$O(n)$
insert	$O(n)$	$O(n)$
delete	$O(n)$	$O(n)$

4. Size Nasıl Artıyor?

- 2 katına çıkıyor

5. Which of the following operations is not $O(1)$ for an array of sorted data. You may assume that array elements are distinct. Aşağıdakilerden hangisi $O(1)$ değildir?

- (A) Find the ith largest element
- (B) Delete an element
- (C) Find the ith smallest element

- (D) All of the above
- (B)

Array Python Problem

- Brute force çözmeye çalışın yani kağıt kalem düz mantık for döngüleri filan kullanara, metot kullanmadan.
- Eğer soruyu çözemezseniz bırakın 1 gün sonra tekrar bakın
- Array Python Challenges/Problems
 1. Word Split

1) Word Split

```
* input = ["deeplearning", "d,dll,a,deep,dee,base,lear,learning"]
* output = ["deep,learning"]
```

In [5]:

```
def wordSplit(liste):
    word = list((liste[0])) # "deeplearning" => ["d","e"...]
    d = liste[1].split(",") # ["d","dll","a","deep","dee","base","lear","learning"]

    for i in range(1,len(word)):
        c = word[:i]
        c.insert(i," ")

        x , y = "".join(c).split() # "d", "eeplearning"      "de","eplearning"
        if x in d and y in d:
            return x + " , " + y
    return "bulamadim (no way)"
```

In [6]:

```
print(wordSplit(["deeplearning", "d,dll,a,deep,dee,base,lear,learning"]))
print(wordSplit(["deeplear2ning", "d,dll,a,deep,dee,base,lear,learning"]))
print(wordSplit(["deeplear2ning", "d,dll,a,deep,dee,base,lear,lear2ning"]))
```

```
deep , learning
bulamadim (no way)
deep , lear2ning
```

Neler Öğrendik

- Arrays
- Dynamic Array
- Dynamic Array with Python
- Array İş Mülakatları Soru-Cevap
- Array Python Challenge/Problem
- TAVSİYE: Array ile ilgili soru çözün. Mesela <https://coderbyte.com/> (<https://coderbyte.com/>)